웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202210641 이름: 장현욱 제출일: 26.10.28

서비스 이름 치이카와 인어섬의 비밀 게시판

기술 조합과 버전 Vue 3.5.43 (Vite 8.3.4) / Spring Boot 3.5.16 (JJWT 0.13.0, sqlite-jdbc 3.51.3.0) / macOS 27.2, Node.js v26.5.0, JDK 21.0.12.1

GitHub 저장소 URL **https://github.com/duriadda/CRUD_login_HW.git**

소스 코드 기준 main 브랜치 / 코드 구현 기준 커밋 ba004e6 (이후 커밋은 README, 테스트 기록, 증거 추가이며 backend/src, frontend/src 변경 없음)

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

얼마 전 개봉한 치이카와 인어섬의 비밀을 본 관람객들과, 기존에 치이카와 원작 만화를 봐왔던 사람들이 여러가지 이야기를 나누고 떡밥에 관한 비밀을 얘기하는 공간입니다. Vue + Spring Boot 를 고른 이유는 초급 프로젝트 수업때 사용해봤기 때문에 더 알아보고 싶어서 선택했습니다. 템플릿에서는 인증 TODO 구현, 화면 주제와 문구 .env 로딩 설정을 건드렸습니다.
application.properties 에 설정 한 줄을 추가했습니다. 나머지 라우트, 입력 검증, 저장소, 계정 코드는 제공된 그대로 사용했습니다.

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.list(), get() / App.vue	로그인 없이 조회 (T01)
로그인과 JWT 발급	완료	BoardController.login(), AuthService.issueToken() / api.js login()	HS256, 기본 TTL 90 초 (T02, T03)
인증과 내 정보 조회	완료	AuthService.currentUserId(), BoardController.me()	401 code 3 종 구분 (T04, T09, T10, T11)
본인 글 작성·수정·삭제	완료	BoardController.create() / update() / delete(), AuthService.assertOwner()	타인 글 403, 작성자 필드 위조 400 (T05 ~ T08, T12, T17)
만료·로그아웃·오류 표시	완료	api.js request() , clearSession(), App.vue describe()	로그인은 탭 메모리에만 보관, 새로고침 시 해제(T10, T11, T16)
DB 데이터 유지	완료	PostStore, backend/data/board.sqlite	서버 재시작 후 유지 (T15)

구조와 변경 내역

브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.

브라우저는 4173 번 포트의 빌드 화면에서 모든 API 를 /api 로 요청하고, Vite preview 의 프록시가 이걸 8000 번 Spring Boot 서버로 전달함. 백엔드는 BoardController 가 요청을 받아 인증이 필요한 경우 AuthService 를 호출, 게시글은 PostStore 를 통해 SQLite 파일에 저장함
로그인 요청은 아직 토큰이 없으므로 Authorization 헤더 없이 이메일과 비밀번호만 보냄. 서버는 계정을 확인한 뒤 토큰을 돌려주고, 프론트는 이 토큰을 api.js 의 session 변수에만 저장함.
게시글 수정 요청부터는 authorizationHeaders()가 저장된 토큰을 Authorization: Bearer <JWT>로 붙임. 서버는 이 토큰을 검증해 얻은 사용자 ID 로 작성자인지 확인함. 토큰은 /me 와 글 등록, 수정, 삭제 요청에만 실리고, 공개 API 인 목록, 상세 조회와 로그인 요청에는 실리지 않음

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend/.../AuthService.java 생성자	없음	설정 주입, 키 형식 검사, 32 바이트 키 변환, 검증기 생성	잘못된 키로 서버가 뜨지 않게 (AI 초안 그대로 반영, 테스트로 검증)
AuthService.issueToken()	501 NOT_IMPLEMENTED	HS256 JWT 발급 (sub, iss, aud, iat, exp)	BE-01 (AI 초안 후 수정 (aud 단일 문자열))
AuthService.currentUserId()	데모 ID 1 반환	Bearer 추출 -> 서명, 알고리즘, 만료, iss, aud 검증 -> exp, sub 계정 존재 확인	BE-02 (AI 초안 그대로 반영, 테스트로 검증)
AuthService.assertOwner()	빈 함수	다르면 403 FORBIDDEN	BE-03 (AI 초안 그대로 반영)
backend/.../application.properties	설정값 기본값	Spring.config.import=optional:file:.env[.properties] 추가	Spring 은 .env 를 읽지 않으므로, clone 후 .env 만 만들면 실행되게함 (AI 제안 반영)
forntend/src/api.js 4 개 함수	TODO, canEdit 은 항상 true	토큰 메모리 보관, Bearer 헤더, 401 시 세션 해제, 작성자 비교	FE-01~04 (AI 초안 그대로 반영)
frontend/src/App.vue	시작 화면, 데모 배너	주제 문구, code 별 안내, 비로그인 안내, 화면 내 삭제 확인, 로그아웃 상태 “내 정보 조회”	FE-03, 04 T11 수행 (AI 초안, T11 중 AI 가 수정)
Frontend/src/style.css, index.html	기본 스타일, 제목	색상, 제목 변경	주제 반영 (AI 초안 작성)

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend/env.example.txt, 각 폴더 README.md	시작 코드 안내	.env 사용 안내로 수정	실제 실행 방법과 일치 (AI 작성)

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

폴더 이름은 frontend-vue/ -> frontend/, backend-spring/ -> backend/로 바꾸었고, 선택하지 않은 템플릿 4 개는 저장소에 넣지 않았음. 테스트 기록(docs/test-result.md), 증거(docs/evidence/), AI 활동 기록(docs/ai-usage-log.md)은 보조 파일로 추가함

로컬 실행 기록

macOS 27.2, Node.js v26.5.0, JDK 21.0.12.1 / 새 폴더 ~/Desktop/clone-test 에 git clone / backend/.env 생성(64 자리 hex JWT_SECRET, .env 는 커밋 안 함) / 백엔드 ./mvnw -q -DskipTests package → java -jar target/jwt-board-1.0.0.jar (8000 번) / 프론트 npm ci → npm run build → npm run preview / DB backend/data/board.sqlite

4173 번에서 빈 DB 에 Alice 로 글 등록 → 201, Location: /api/posts/1 / T15 에서 서버 Ctrl+C 후 재실행해도 17 번 글의 내용·작성자·시각이 같음 / 증거: CLONE_*.png, T15_restart.txt, 자세한 재현 방법은 README

핵심 코드 설명

3 1 로그인과 JWT 발급

backend/src/main/java/edu/classroom/board/BoardController.java login() (63~69 행), AuthService.java issueToken() (52~63 행)

코드 발췌

```
// BoardController.login()

var input = fields(body, Map.of("email", 254, "password", 128));

var user = users.verify(input.get("email"), input.get("password"));

if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");

return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);



// AuthService.issueToken()

return Jwts.builder()

  .subject(String.valueOf(user.id()))

  .issuer(issuer)

  .audience().single(audience)

  .issuedAt(Date.from(now))

  .expiration(Date.from(now.plusSeconds(ttl)))

  .signWith(key, Jwts.SIG.HS256)

  .compact();
```

동작과 응답 코드 설명

[입력 검증, 계정 확인, 토큰 클레임과 유효 시간, 응답을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

입력: 본문 필드 2 개와 길이 제한 근거: login() 64 행 Map.of(“email”, 254, “password”, 128)

처리
1. fields()가 허용 필드만, 문자열인지, 길이가 맞는지 검사 -> 아니면 400 반환
2. Users.verify()가 PBKDF2 해시로 비밀번호 비교, 계정이 없어도 기준 계정으로 해시 계산을 똑같이 하므로, 응답 시간 차이로 가입 여부를 알아내긴 힘듦
3. 통과한 사용자만 issueToken()이 JWT 를 만듦. 클레임 5 개(sub, iss, aud, iat, exp)뿐이고, 비밀번호나 해시는 넣지 않음. 서명은 HS256 이며, 키는 .env 의 JWT_SECRET 을 32 바이트로 바꾼 값임. 생성자는 키가 비었거나 예시 값이거나 형식이 틀리면 예외를 던져 서버 시작 자체를 막음

출력: 200 과 {accessToken, expiresIn, user}를 돌려줌. user 에는 id, name, email 만 들어있음

실패 조건: 필드 누락, 추가 필드, 형식, 길이 오류는 400 BAD_INPUT 임. 없는 이메일과 틀린 비밀번호는 모두 401 INVALID_CREDENTIALS 이고 메시지도 같음. 둘을 구분해 알려주면 어떤 이메일이 가입되어 있는지 드러나기 때문

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend/src/api.js session, authorizationHeaders(), request(), login()

코드 발췌

```
let session = { accessToken: "", user: null };

function authorizationHeaders() {

 return session.accessToken

  ? { Authorization: `Bearer ${session.accessToken}` }

  : {};

}

// request() 안

if (response.status === 401) clearSession();

// login() 안

session = { accessToken: data.accessToken, user: data.user };
```

동작과 응답 코드 설명

저장 위치: login()은 로그인 응답을 받으면 토큰과 사용자 정보를 모듈 변수인 session 에 저장함. localStorage 나 sessionStorage 는 쓰지 않음. XSS 취약점이 생겼을 때 위험하기 때문
대신 새로고침하면 변수가 초기화되어 로그인이 풀림

헤더 첨부: 모든 API 요청은 request()를 거치고, 18 행에서 authorizationHeaders()의 결과를 헤더에 펼쳐 넣음. 이 함수는 토큰이 있으면 Authorization: Bearer <토큰>을, 없으면 빈 객체 {}를 돌려줌. 그래서 토큰이 없을 때는 헤더 자체가 빠지고 Bearer undefined 같은 잘못된 값이 나가지 않음.
로그인 후 보낸 /me 와 글 등록 요청에는 헤더가 들어있었고, 로그아웃 요청에는 Authorization 줄이 없었음

어느 요청에 붙는가: 로그인 상태에서 보내는 /me, 글 등록, 수정, 삭제 요청에 실림. 목록, 상세 조회는 서버가 토큰을 요구하지 않으므로 비로그인 상태에서도 200 임

401 처리: 응답이 401 이면 request()가 clearSession()으로 메모리의 토큰과 사용자를 지움
401 은 토큰이 없거나 변조, 만료되어 더 이상 쓸 수 없다는 뜻이기 때문임. 그러면 화면이 로그인 폼으로 돌아가고 code 에 맞는 안내가 뜸

로그아웃: App.vue 의 logout()은 clearSession()을 호출하고 화면 입력을 초기화함. 서버에는 요청을 보내지 않고, 브라우저 메모리의 상태만 지움

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend/src/main/java/edu/classroom/board/AuthService.java currentUserId(), 검증기 생성

코드 발췌

```
if (header == null || !header.matches("Bearer [^\\s]+"))
 throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
try {
 var jws = parser.parseSignedClaims(token);   // 서명·iss·aud·만료 검증
 if (!"HS256".equals(jws.getHeader().getAlgorithm()))
  throw new ApiError(401, "INVALID_TOKEN", "허용되지 않은 토큰 알고리즘입니다.");
 claims = jws.getPayload();
} catch (ExpiredJwtException e) {
 throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요.");
} catch (JwtException | IllegalArgumentException e) {
 throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
}
if (claims.getExpiration() == null) throw new ApiError(401, "INVALID_TOKEN", ...);
if (sub == null || !sub.matches("[1-9][0-9]{0,15}")) throw new ApiError(401, "INVALID_TOKEN", ...);
if (users.find(id) == null) throw new ApiError(401, "INVALID_TOKEN", ...);
return id;
```

동작과 응답 코드 설명

입력: 보호 API 요청의 Authorization 헤더

처리
1. 헤더 형식: Bearer <공백 없는 값> 형태가 아니면 토큰이 없는 것으로 보고 401 TOKEN_REQUIRED 를 던짐. 헤더 없음, Basic..., Bearer 뒤가 빈 경우가 여기서 갈림
2. 라이브러리 검증: 생성자에서 미리 만든 parser 가 서명, 발급자, 사용 대상, 만료를 한 번에 검증함. 예외는 두 갈래로 나누는데, JJWT 는 서명을 먼저 확인한 뒤 만료를 검사하므로, ExpiredJwtException 이 났다면 서명은 정상이고 시간만 지난 토큰임. 그래서 이 경우만 TOKEN_EXPIRED 로 분류하고, 나머지 오류는 모두 INVALID_TOKEN 으로 처리
3. 직접 추가한 검사: JJWT 는 exp 가 아예 없는 토큰도 통과시키므로 exp 가 있는지 직접 확인함. sub 가 양의 정수 문자열인지, 그 ID 의 계정이 실제로 있는지도 확인함. 하나라도 아니면 INVALID_TOKEN 임

출력: 세 단계를 모두 통과한 사용자 ID 만 반환함. me(), create(), update(), delete()는 이 값만 현재 사용자로 쓰고, 요청 본문에 담긴 값은 쓰지 않음

실패 응답
401 TOKEN_REQUIRED | 헤더 없음, 형식 오류 | 근거: T04, T11
401 INVALID_TOKEN | 서명 변조, 알고리즘 iss, aud, exp, sub 오류, 없는 사용자 | 근거: T09
401 TOKEN_EXPIRED | 정상 서명의 만료 토큰 | T10

제공된 Errors.java 가 모든 401 응답에 WWW-Authenticate: Bearer 를 붙임. 로컬 키로 클레임을 조작한 토큰 15 종도 모두 기대한 code 로 거부됨

3 4 작성자 권한 검사

파일 / 함수: AuthService.java assertOwner(), BoardController.java create(), update(), delete()

코드 발췌

```
// BoardController.update()

long user = auth.currentUserId(req), pid = postId(id);   // ① 인증

var p = existing(pid);                          // ② 글 존재 (404)

auth.assertOwner(user, p.authorId());                  // ③ 작성자 권한 (403)

var input = fields(body, Map.of("title", 100, "content", 5000)); // ④ 입력 검증 (400)

return store.update(pid, input.get("title"), input.get("content")); // ⑤ DB 변경



// AuthService.assertOwner()

if (userId != authorId)

  throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");



// BoardController.create()

long user = auth.currentUserId(req);

var p = store.create(input.get("title"), input.get("content"), user);
```

동작과 응답 코드 설명

비교 대상: assertOwner(user, p.authorId())는 두 값을 비교함. user 는 currentUserId()가 토큰을 검증해 얻은 현재 사용자 ID 이고, p.authorId()는 DB 에 저장된 글의 작성자 ID 임. 둘 다 클라이언트가 요청 본문으로 바꿀 수 없는 값이라는 점이 핵심. 다르면 403 FORBIDDEN 을 던짐

처리 순서: 수정, 삭제는 인증 -> 글 존재 -> 작성자 권한 -> 입력 검증 -> DB 변경 순서로 처리하고, 앞 단계에서 걸리면 뒤 단계는 실행하지 않음. T14 에서 같은 “없는 글 수정”이라도 토큰이 없으면 401 TOKEN_REQUIRED, 정상 토큰이면 404 NOT_FOUND 가 나옴.
같은 ‘공백 제목’이라도 Bob 이 Alice 글에 보내면 403 FORBIDDEN, Alice 가 자기 글에 보내면 400 BAD_INPUT 이였음. 권한이 없는 요청은 DB 변경 순서에 도달하지 못하므로 DB 가 바뀌지 않음

401 과 403 의 차이: 401 은 ‘누구인지 확인하지 못함’ 토큰없음, 변조, 만료라 다시 로그인 해야함. 403 은 ‘누군지 확인했으나, 글의 작성자가 아님’이라 토큰은 정상임. 그래서 프론트는 401 이면 로그인 상태를 지우고, 403 이면 로그인을 유지한 채 안내만 함

작성자 위조 방지: create()는 작성자를 요청 본문이 아니라 currentUserId()의 결과 user 로 저장함. T05 에서 본문에 작성자를 넣지 않았는데도 authorId:1 로 저장됨. 또 fields()가 title, content 외의 필드를 거부하므로, 본문에 authorId 나 userId 를 넣으면 400 BAD_INPUT 이 되고 글이 생성되지 않음

화면과의 관계: 프론트의 canEdit()은 로그인 사용자와 작성자가 같을 때만 수정, 삭제 버튼을 보여주는 표시용 판단. T07 의 Bob 화면에는 Alice 글의 버튼이 없었지만, curl 로 직접 요청하자 서버의 assertOwner()가 403 으로 막았음. 실제 보안 경계는 서버 검사

테스트 결과

실행 환경: macOS 27.2 / Node.js v26.5.0, JDK 21.0.12.1 / Chrome 155 / 백엔드 java -jar target/jwt-board-1.0.0.jar (backend 폴더), 프론트 npm run build → npm run preview (http://127.0.0.1:4173)

로컬 배포 확인: GitHub 저장소를 새 폴더 ~/Desktop/clone-test 에 clone 하고 README 대로 .env 생성 -> ./mvnw -q -DskipTests package -> java -jar ... -> npm ci -> npm run build -> npm run preview 순서로 실행함. 4173 에 접속해서 빈 DB 에 Alice 로 글을 등록하니 201 Created, Location: /api/posts/1 이 나왔음

테스트 일시 및 코드 버전: 2026-10-26 ~ 2026-10-27, 코드 기준 커밋 ba004e6

토큰 TTL: 일반 테스트 90 초 / 만료 테스트 3 초 (TOKEN_TTL = 3 으로 서버 재시작, T10)

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인, 목록, 9 번 상세 조회(화면)	200	200 / 해당없음 x2, Authorization 없음	PASS	T01_posts_200-1, 2, T01_9_200-1, 2
T02	Alice 로그인 (화면)	200, 토큰-user	200 / 해당없음, expiresin 90, user id 1	PASS	T02_login_200, T02_login_response, T02_screen_logged_in
T03	틀린 비밀번호, 없는 이메일 (화면)	401 INVALID_CREDENTIALS	401/INVALID_CREDENTIALS x2, 토큰 없음	PASS	T03_wrong_pw_401·response·screen, T03_no_email_401·response
T04	토큰 없이 /me 글 등록 (curl)	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED x2 글 수 3->3	PASS	T04_curl.txt, T04_screen_login_guide
T05	Alice 글 등록 후 /me (화면)	201, authored 1 / 200	201 / 해당 없음 (Location/api/posts/12, authored 1), 200 / 해당 없음	PASS	T05_create_201, T05_create_201_Authorization, T05_create_response, T05_me_200, T05_screen
T06	Alice 본인 12 번 수정, 삭제(화면)	200, 204	200 / 해당 없음 (createdAt 유지), 204 / 해당 없음 (본문 없음)	PASS	T06_update_200·response, T06_delete_204·response_empty, T06_screen_deleted
T07	Bob 토큰으로 Alice 9 번 수정 (curl)	403 FORBIDDEN	403 / FORBIDDEN , 원문 동일	PASS	T07_T08_curl.txt, T07_screen_bob_no_buttons

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T08	Bob 토큰으로 Alice 9 번 삭제 (curl)	403 FORBIDDEN	403 / FORBIDDEN , 글 유지	PASS	T07_T08_curl.txt
T09	Alice 토큰 서명 첫 글자 변조, /me POST, PUT, DELETE (curl)	401 INVALID_TOKEN	401 / INVALID_TOKEN x4 대조군 200, 데이터 동일	PASS	T09_curl.txt
T10	TTL 3 초, 만료 후 /me PUT, DELETE (curl), /me (화면)	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED x4 화면 로그아웃	PASS	T10_curl.txt, T10_me_401·response, T10_screen_logged_out
T11	로그아웃 후 /me (화면)	401 TOKEN_REQUIRED	401 TOKEN_REQUIRED, Authorization 없음	PASS	T11_me_401_no_auth, T11_me_response, T11_screen
T12	Bob 등록에 authorId:1 / userId:1 (curl)	400 BAD_INPUT	400 / BAD_INPUT ×2, 위조 글 미생성	PASS	T12_curl.txt
T13	빈 제목·공백·101 자·5001 자 등 7 종 (curl), 공백 제목 (화면)	400 BAD_INPUT	400 / BAD_INPUT ×8, 입력값 유지	PASS	T13_curl.txt, T13_ui_400, T13_screen_input_kept
T14	없는 99999 조회·수정·삭제, abc·0·-1, 겹침 4 종 (curl), 삭제된 글 삭제 (화면)	404 / 400	404 / NOT_FOUND ×3(+화면 1), 400 / BAD_INPUT ×3, 겹침 401·404·403·400	PASS	T14_curl.txt, T14_ui_404, T14_screen_404
T15	17 번 등록 → 서버 Ctrl+C → 재실행 → 재조회	200, 글 유지	200 / 해당 없음, 내용·작성자·시각 동일	PASS	T15_restart.txt, T15_screen_after_restart
T16	Alice 로그인 후 새로고침 (화면)	글 유지, 로그인 해제	목록 6 개 유지, 로그인 폼으로 복귀	PASS	T16_before_refresh, T16_after_refresh
T17	무토큰 PUT·DELETE, Bob 본인 글 작성·수정·삭제 (curl)	401 / 201·200·204	401 / TOKEN_REQUIRED ×2, 201·200·204 / 해당 없음, 삭제 후 404	PASS	T17_curl.txt

HTTP 응답 확인 기록

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 (목록·상세 T01, 로그인 T02, /me T05, 수정 T06·T17)	T01_*, T02_login_200, T06_update_200
새 글 등록	**201 및 Location**	201, Location: /api/posts/12 (T05), /api/posts/18 (T17), /api/posts/1 (clone)	T05_create_201, T17_curl.txt, CLONE_screen_working
본인 글 삭제	**204 및 빈 본문**	204, Response 탭 본문 없음 (T06), curl 본문 줄 없음 (T17)	T06_delete_204, T06_delete_response_empty
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT (T12 2 건, T13 8 건, T14 잘못된 ID 3 건)	T12_curl.txt, T13_curl.txt, T14_curl.txt
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS (T03 2 건)	T03_wrong_pw_response, T03_no_email_response
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED (T04, T11, T14, T17)	T04_curl.txt, T11_me_response
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN (T09 4 건)	T09_curl.txt
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED (T10 curl 3 건, 화면 1 건)	T10_curl.txt, T10_me_response
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN (T07, T08, T14 겸침)	T07_T08_curl.txt
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND (T14 curl 3 건 + 화면 1 건, T17 삭제 후)	T14_curl.txt, T14_ui_404

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

보호 API 의 401 응답에는 모두 WWW-Authenticate: Bearer 가 있었음.(T04, T09, T11, T17 curl 출력, T10_me_401, T11_me_401, T11_me_401_no_auth 캡처) 403 응답에는 없었음.

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

401 일 때: api. js 가 메모리의 토큰과 사용자를 모두 지워서 로그인 카드가 로그인 폼으로 돌아가고, 리뷰 쓰기 칸이 “리뷰를 작성하려면 먼저 로그인하세요.” 로 바뀜. 안내 문구는 code 에 따라 달라짐

403 일 때: App.vue 의 codeMessages.FORBIDDEN 문구 “본인이 작성한 리뷰만 수정, 삭제 할 수 있습니다.” 를 띄우고, 로그인 상태와 입력중인 내용은 그대로 둠. 다만 화면에서는 타인 글의 수정, 삭제 버튼이 숨겨져있어 403 화면으로는 나타내지 못함. 서버의 403 응답은 curl 로 확인

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

T04(글 수 3->3), T07, T08 (9 번 원문 동일), T09(13 번 원문, 글 수 동일), T10(10 번 원문 동일), T12(목록 동일), T13(글 수 4->4), T14(13 번 원문 동일), T17(10 번 원문 동일), 모두 실패 요청 전후 공개 조회 결과가 updatedAt 까지 같았음

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

각각 401/TOKEN_REQUIRED, 404/NOT_FOUND(T14_curl.txt). 추가로 Bob 이 Alice 글에 공백 제목 -> 403/FORBIDDEN, Alice 가 자기 글에 공백 제목 -> 400/BAD_INPUT.

웹서비스설계 04 계속

테스트 증거

로그인과 토큰 전송

사진이 계속 잘려서 파일 이름으로 대신합니다. (CLONE_screen_working.png)

다른 작성자의 수정과 삭제

T07_T08_curl.txt 에서 가져옴

Bob 확인 (GET /api/me, Authorization: Bearer <TOKEN>) {"id":2,"name":"Bob","email":"bob@class.test"} ### T07: Bob 이 Alice 글 수정 (PUT /api/posts/9) HTTP/1.1 403 {"message":"본인이 작성한 글만 수정·삭제할 수 있습니다.","code":"FORBIDDEN"} ### T08: Bob 이 Alice 글 삭제 (DELETE /api/posts/9) HTTP/1.1 403 {"message":"본인이 작성한 글만 수정·삭제할 수 있습니다.","code":"FORBIDDEN"} (요청 전후 GET /api/posts/9 결과 동일)

토큰 변조와 만료

T09 변조 (서명 첫 글자 T→A): GET /api/me, POST, PUT, DELETE → 모두 HTTP/1.1 401, WWW-Authenticate: Bearer, "code":"INVALID_TOKEN" T10 만료 (TTL 3 초, 5 초 대기): GET /api/me, PUT, DELETE → 모두 HTTP/1.1 401, WWW-Authenticate: Bearer, "code":"TOKEN_EXPIRED"

자동 검사와 로컬 배포 증거

제공된 tools/check_api.py 는 핵심 API 의 상태 코드와 code, 실패 후 데이터 불변을 자동으로 확인하는 도구. –mode final 은 40 개, TOKEN_TTL=3 에서 –expiry 를 더하면 만료 검사까지 48 개 항목을 모두 통과함. 화면 처리와 서버 재시작은 이 도구가 확인하지 않으므로 T01~T17 로 따로 확인함

AI 활용과 참고 자료

AI 사용 여부: [사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude Code (VS Code 확장) / Claude Opus 5.5	설계 (과제 분석·구현 계획)	작업 순서	참고
〃	코드 생성	AuthService.java, application.properties, api.js, App.vue, style.css, index.html	그대로 반영 (aud 는 검증 중 수정)
〃	테스트 (절차, curl 명령 작성, 자동 검사 실행)	T01 ~ T17 절차, docs/evidence/*.txt	수정 후 반영 (명령 오류 2 건 수정)
〃	디버깅 (T11 수행불가 UI 수정)	App.vue 로그아웃 상태 "내 정보 조회" 버튼	수정 후 반영
〃	문장 정리 (기록, README)	README.md, docs/test-results.md, docs/ai-usage-log.md	수정 후 반영
〃	보고서 문장 정리	이 보고서의 1~6 절 서술 초안, 4.1 로컬 실행 기록	수정 후 반영

대표 사례

도구 / 날짜: Claude Code, Claude Opus 5.5 / 2026-10-26

요청한 내용: 과제 공지 PDF 를 읽고, 해결 계획을 세운 뒤 Vue+ Spring Boot 조합으로 웹 게시판 초안 작성을 요청

AI 가 제안한 내용: AuthService 의 발급, 검증, 권한 함수와 서버 시작 시 키 검사, .env 를 읽는 설정 한 줄, api.js 의 토큰 보관, 전송, 401 처리, App.vue 의 화면 상태와 code 별 안내 작성

실제 반영 위치: AuthService.java 전체, application.properties 1 줄, api.js, App.vue, style.css, index.html

내가 확인하고 수정한 내용: 코드는 직접 고치지 않고 AI 초안을 그대로 반영. 대신 제공 코드와 함께 명세대로 동작하는지 테스트로 확인함. 검증 중 발급 토큰의 aud 가 배열로 나와서 AI 가 명세 표기에 맞게 단일 문자열로 바꿈. 또 T11 을 하려는데 로그아웃 상태에서 "내 정보 조회" 버튼이 숨겨져있어 요청을 보낼 수 없었고, AI 가 로그아웃 상태에도 버튼이 보이도록 App.vue 를 수정

검증 방법과 결과: 직접 수행한 테스트 결과로 그대로 반영한 코드를 믿었음. T01~T17 전 항목에서 기대한 HTTP 상태와 code 가 나옴. 특히 실패해야 하는 요청은 모두 거부됨. 타인 글 변경은 403, 서명 변조는

401 INVALID_TOKEN, 실제 만료는 401 TOKEN_EXPIRED, 작성자 위조는 400 이었고, 거부 뒤 재조회했을 때 데이터가 바뀌지 않았음. 추가로 AI 가 실행한 자동검사와 조작 토큰 15 종 검사도 통과함

도구 / 날짜: Claude Code / 2026-10-27

요청한 내용: T09 테스트를 진행할 명령 요청

AI 가 제안한 내용: 정상 토큰의 서명 첫 글자를 바꾸는 Python 한 줄을 포함한 curl 스크립트를 주었음

실제 반영 위치: docs/evidence/T09_curl.txt

내가 확인하고 수정한 내용: 처음 실행했을 때 기대한 INVALID_TOKEN 이 아니라 네 요청 모두 401 TOKEN_REQUIRED 가 나옴. 해당 코드는 헤더가 없거나 Bearer 뒤가 비어있을 때만 나오는 코드이므로, 서버가 아니라 보낸 토큰을 의심함. 확인해보니 변조 토큰을 만드는 Python 명령에 문법 오류가 있어, 변조 토큰이 빈 문자열이 되어 전송되고 있었음. 서버 응답은 명세대로였지만, 서명 변조를 시험한 것은 아니므로 첫 결과는 버림. 고친 명령에는 변조가 실제로 일어났는지 보이도록 “서명 첫 글자 T->A, 길이 183/183”을 출력하는 줄을 넣었음

검증 방법과 결과: 고친 명령으로 다시 실행해, 대조군의 /me 는 200 이고 변조 토큰으로 보낸 GET, POST, PUT, DELETE 는 모두 401 INVALID_TOKEN 임을 확인함. 13 번 글과 전체 글 수는 요청 전후로 같았음. 첫 시도의 결과는 오히려 서버가 “토큰 없음”과 “잘못된 토큰”을 실제로 구분한다는 부수적인 증거가 됨

도구 / 날짜: Claude Code / 2026-10-27

요청한 내용: T16(로그인 후 새로고침) 캡처를 검토하고 기록해 달라고 요청

AI 가 제안한 내용: 새로고침 후 이메일 칸에 보이는 alice@class.test 를 "브라우저가 입력값을 복원한 것"이라고 기록

실제 반영 위치: docs/test-results.md T16

내가 확인하고 수정한 내용: AI 가 새로고침 후 캡처의 이메일 칸에 보이는 alice@class.test 를 브라우저가 입력을 복원한 것이라고 기록했는데, 입력값이 아니라 예시 문구일 수 있다고 지적함. 확인해보니 App.vue 이메일 입력칸에 placeholder=alice@class.test 가 있었음. 캡처의 글자는 같은 화면의 다른 예시 문구와 같은 회색이었고, 실제 남아있던 T10 캡처의 이메일은 검정이었음.

검증 방법과 결과: 글자 색을 비교해 placeholder 가 맞음을 확인하고 test-results.md 의 T16 기록을 정정함.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
수업 제공 시작 템플릿 frontend-vue, backend-spring	프로젝트 전체	라우트, 저장소, 계정, 화면 골격을 그대로 사용
수업 제공 docs/02_API 명세.md, tools/check_api.py, 05_테스트시나리오.md	API 응답 규칙, 자동 검사, T01 ~ T17	명세대로 구현, 검증

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
JJWT https://github.com/jwtk/jjwt	AuthService.java	Jwts.builder(), Jwts.parser().verifyWith() 사용법
Spring Boot 외부 설정 https://docs.spring.io/spring-boot/3.5/reference/features/external-config.html	Application.properties	Spring.config.import 로 .env 읽기
Vite 가이드 https://vite.dev/guide/	Vite.config.js 빌드, preview	프록시와 로컬 빌드 실행

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문은 클라이언트가 마음대로 만들 수 있음. 서버가 본문의 authorId 를 그대로 저장하면, Bob 이 “authorId”: 1 을 넣어 Alice 이름으로 글을 쓸 수 있음. 그 글은 DB 에 Alice 글로 남으므로, 이후 권한 검사도 그 잘못된 작성자를 기준으로 하게됨. 내 코드는 이걸 두 겹으로 막음
1. BoardController.create()는 작성자를 본문이 아니라 currentUserId()가 토큰을 검증해 얻은 ID 로 정함
2. fields()가 title, content 외의 필드를 허용하지 않아 authorId 나 userId 가 들어오면 400 BAD_INPUT 으로 요청 자체를 거부함. T12 에서 Bob 토큰으로 authorId: 1 과 userId: 1 을 각각 넣어 보냈을 때 둘 다 400 이었고, 글은 생성되지 않음. T05 에서는 본문에 작성자 없이 보냈는데도 authorId: 1 로 저장됨

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼은 브라우저 화면에만 있는 것이고, API 는 화면 없이도 호출할 수 있음. 토큰만 있으면 curl 이나 API 클라이언트로 PUT /api/posts/9 를 직접 보낼 수 있고, 개발자 도구에서 화면 코드를 바꿔 숨긴 버튼을 다시 보이게 할 수도 있음. 그래서 프론트의 canEdit()는 사용자 편의를 위한 표시일 뿐임. 실제 권한 검사는 서버의 AuthService.assertOwner()가 진행. 이 함수는 토큰에서 검증한 사용자 ID 와 DB 에 저장된 글의 authorId 를 비교해 다르면 403 FORBIDDEN 을 던짐. T07 에서 Bob 화면에는 Alice 글의 수정·삭제 버튼이 없었지만, 같은 Bob 토큰으로 curl 요청을 보내자 서버가 403 으로 막았고 글은 그대로였음.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

JWT 의 헤더와 페이로드는 Base64URL 로 인코딩만 되어 있어 누구나 풀어서 읽을 수 있음. decode 한 내용은 누군가 sub 를 다른 ID 로 바꿔 넣었을 수도 있으므로 그대로 믿으면 안 됨. verify 는 서버만 아는 키로 서명을 다시 계산해 토큰의 서명과 같은지 확인하는 것. 내용이 한 글자라도 바뀌면 서명이 맞지 않으므로 위조를 알아낼 수 있음. 내 코드의 currentUserId()는 다음을 검증

JJWT parser 로 서명, iss, aud, 만료를 검증

헤더 알고리즘이 HS256 인지 확인

라이브러리가 검사하지 않는 exp 존재, sub 형식, 실제 계정 존재를 직접 확인

T09 에서 서명 첫 글자 하나만 바꿔도 401 INVALID_TOKEN 이 나옴

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃 | 브라우저 메모리(session) | 토큰과 사용자를 지우고 화면을 로그인 폼으로 바꿈. 서버에는 요청을 보내지 않고, 이후 요청에는 Authorization 헤더가 붙지 않음

토큰 만료 | 서버의 판단 | exp 시각이 지나면 서버가 같은 토큰을 401 TOKEN_EXPIRED 로 거부함. 프론트는 이 401 을 받은 뒤 메모리 상태를 지우고 재로그인 안내

새로고침 | 브라우저 JavaScript 전체 | 페이지가 다시 시작되면서 메모리 변수가 초기화되어 로그인 사라짐. 게시글은 서버의 SQLite 파일에 있으므로 그대로 남음

이미 다른 곳에 복사된 토큰은 로그아웃해도 무효가 되지 않음. 서버는 발급한 토큰 목록을 저장하지 않고 서명과 만료만 확인하므로, 그 토큰은 exp 시각까지 유효함. 기본 TTL 을 90 초로 짧게 두는 것이 이 위험을 줄이는 방법

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

T09 서명 변조 테스트에서 기대한 INVALID_TOKEN 대신 TOKEN_REQUIRED 가 나온 일이 가장 헷갈렸다.
처음에는 서버 검증 코드가 잘못된 줄 알았지만, 출력을 보니 네 요청 모두 같은 code 였다.
TOKEN_REQUIRED 는 헤더가 없거나 Bearer 뒤가 비어 있을 때만 나오는 code 라서, 서버보다 보낸 토큰 쪽을 의심했다. 확인해 보니 변조 토큰을 만드는 Python 명령이 문법 오류로 실패해 빈 문자열이 전송되고 있었다.
명령을 고치고, 변조가 실제로 일어났는지 보이도록 서명 첫 글자와 토큰 길이를 출력하는 줄을 추가했다. 다시 실행하자 "T -> A, 길이 183 / 183"이 확인됐고 네 요청 모두 401 INVALID_TOKEN 이 나왔으며, 요청 전후 데이터도 같았다. 이 일로 테스트가 실패했을 때 서버만 볼 것이 아니라 테스트 입력이 의도대로 만들어졌는지도 확인해야 한다는 것을 배웠다.

현재 남은 문제와 다음에 개선할 점:

이미 발급된 토큰을 서버에서 무효화하지 못하고, 새로고침하면 로그인이 풀림. 개선하면 Access Token 을 짧게 유지하면서 Refresh Token 으로 다시 발급받거나, 서버에 무효화한 토큰을 목록으로 두는 방법을 생각할 수 있음

제출 확인: [x] 실제 코드와 설명 일치 [x] 테스트 직접 수행 [x] AI 활용 및 출처 기록 [x] 비밀 키와 토큰 제거

GitHub 확인: [x] 루트 README 작성 [x] 보고서 PDF 와 증거 업로드 [x] README 문서 링크 확인 [x] 교수자 접근 가능 [x] 저장소 URL 과 커밋 SHA 제출